

Sapienza Università di Roma  
corso di laurea in Ingegneria informatica e automatica

# **Tecnologie e Sistemi Web**

a.a. 2025/2026

## **Parte 2**

### **CSS**

Lorenzo Marconi

# Fogli di stile

- In HTML non c'è separazione tra **forma** e **contenuto** del documento
- Fogli di stile o Cascading Style Sheets (CSS) = estensione di HTML introdotta da Internet Explorer 3
- I CSS si occupano di gestire la formattazione del documento esternamente al documento stesso
- Standard W3C (CSS1 e CSS2)

# Tipi di fogli di stile

- HTML permette di utilizzare diversi linguaggi per fogli di stile
- Linguaggio standard W3C per i fogli di stile: CSS (Cascading Style Sheets)
- `text/css`

# Tipi di CSS

- CSS in linea
  - agiscono su singole istanze di testo all'interno della pagina
- CSS incorporati
  - agiscono in modo globale su un singolo documento
- CSS esterni
  - agiscono in modo globale su insiemi di documenti

# CSS in linea

Agiscono su singole istanze di testo all'interno della pagina

Esempio:

```
<DIV STYLE="font-size:18px;  
    font-family:arial; color:red">  
    Questa parte di testo ha colore rosso  
</DIV>
```

Marcature più usate:

- senza “semantica” (<DIV> o <SPAN>)
- ricorda: `style` è un attributo globale!

# CSS in linea: pro e contro

- **Semplici** da usare (si possono considerare una “estensione” di HTML)
- **Difficili** da mantenere
- Modificano il contenuto del testo
- Non rispondono all’esigenza di separazione tra forma e contenuto
- È opportuno usarli in modo molto circoscritto

# CSS incorporati

- Agiscono in modo globale su un singolo documento HTML
- Vanno scritti nella parte intestazione del documento (tra <HEAD> e </HEAD>)
- Permettono di dare opzioni di formattazione **globale** ai tag (e quindi al documento)

# CSS incorporati: esempio

```
<HTML>
<HEAD>
<style type="text/css">
  H1 { font-size:17px; color:green; }
  H2 { font-family:arial; color:red; }
</style>
</HEAD>
<BODY>
  <H1>Tutti gli H1 sono di colore verde</H1>
  <H2>Tutti gli H2 sono di colore rosso</H2>
</BODY>
</HTML>
```



# CSS incorporati: sintassi

- Ogni **dichiarazione** CSS è una coppia **selettore – assegnazione di proprietà**
- Dichiarazioni differenti sono separate da un punto e virgola
- Ad esempio:

```
H1 {font-size:17px; color:green;}
```

- H1 è il selettore

- {font-size:17px; color:green;} è l'assegnazione delle proprietà

# CSS incorporati: sintassi

```
H1 { font-size:17px; color:green; }
```

```
H2 { font-family:arial; color:red; }
```

- gli attributi sono inseriti tra parentesi graffe
- al posto del segno = vengono usati i due punti
- gli attributi composti da più termini sono separati da un trattino (kebab-case)
- quando un attributo è considerato proprietà di un oggetto i trattini si eliminano e le iniziali dei termini diventano maiuscole (camelCase)
- esempio: `font-style` diventa `fontStyle`

# Il tag `<style>`

```
<style type="text/css" media="...">
```

- L'attributo `TYPE` del tag `<STYLE>` definisce il **linguaggio** in formato MIME del foglio di stile
- `MEDIA` permette di specificare una **media query**
- Se l'attributo `TYPE` viene omesso, il browser lo identifica di default con `text/css`

**Attenzione:** non confondere il *tag* `<style>` con l'*attributo* `style` usato nei CSS in linea

# CSS incorporati: pro e contro

- I CSS incorporati permettono di configurare uniformemente la formattazione del testo (cioè l'aspetto dei tag) in un documento
- I CSS incorporati sono definiti al di fuori del corpo del documento, e permettono così la **separazione** tra contenuto e forma
- Tuttavia, i CSS incorporati sono inadatti a trattare in modo uniforme **insiemi di documenti** (ad esempio un sito web)

# CSS esterni

agiscono in modo **globale** su insiemi di documenti:

- gli stili dei singoli marcatori vengono raggruppati in un documento separato (file distinto dai documenti)
- ogni documento “richiama” gli stili (con un opportuno comando)
- una modifica sul file di stili genera automaticamente la stessa modifica su tutti i documenti che lo richiamano

# CSS esterni

Esempio: file `stile.css`:

```
H1 { font-size:17px; color:green; }  
H2 { font-family:arial; color:red; }
```

contiene gli stili che si vogliono definire

# Collegamento ad un CSS esterno

Collegamento ad un CSS esterno in un documento:

```
<link rel="stylesheet" href="stile.css" type="text/css">
```

- Il tag `<link>` identifica un file esterno al documento HTML
- L'attributo `rel` indica il tipo di file collegato (stylesheet)
- L'attributo `href` richiama il percorso e il nome del file esterno

# Vantaggi dei CSS esterni

- Permettono la separazione tra contenuto e forma
- Permettono di gestire insiemi di documenti
- Massima flessibilità di utilizzo
- Riutilizzabilità degli stili
- Possibilità di **fondere** insieme più stili (*cascading*)



# Proprietà di stile per il testo

- font-family (serif / sans serif / cursive ....)
- font-size
  - medium / xx-small / x-small / small / large / x-large / xx-large / smaller / larger / [valore personalizzato]
  - punti (pt) / pixel (px) / centimetri (cm) / pollici (in) / percentuale (%), ...
- font-style (normal / italic / oblique)
- font-variant (normal / small-caps)
- font-weight
  - normal / bold / bolder / lighter / [valore personalizzato]
- text-decoration
  - none / overline / underline / line-through

# Esempio

Per i CSS in linea:

```
<A style="text-decoration:none"  
  href="#a1">ciao </a>
```

Per i CSS incorporati:

```
<style>  
  A {text-decoration: none}  
</style>
```

# Versioni di CSS

- Il W3C ha rilasciato varie versioni di CSS:
  - CSS 1 (1996)
  - CSS 2 (1998)
  - CSS 2.1 (2011)
  - CSS 3 (2011-2014)
- Tra le differenze principali delle varie versioni, si segnala il numero di proprietà definite e quindi assegnabili:
  - CSS1 = 53 proprietà
  - CSS2 = 122 proprietà
  - CSS3 = 339 proprietà
- Attualmente, CSS è un living standard composto da moduli, ciascuno dei quali incentrato su aspetti specifici ed eventualmente pubblicato in varie versioni (*livelli*)

# Selettori CSS

- Fino ad ora abbiamo visto selettori CSS corrispondenti a nomi di elementi HTML (che selezionano tutti gli elementi del documento HTML con quel nome)
- In realtà si possono costruire selettori molto più complessi

# Classi

- A tutti gli elementi è possibile assegnare l'attributo `CLASS`
- Questo attributo permette ai fogli di stile di trattare singole occorrenze o sottoinsiemi di occorrenze dello stesso elemento o di elementi diversi

# Classi: esempio

```
<STYLE>
  H2.top { font-family:verdana; font-size:15px;
    font-weight:bold; font-style:normal; }
  H2.bottom { font-family:arial; font-size:10px;
    font-weight:bold; font-style:italic; }
</STYLE>

<H2 class="top">Testo della pagina</H2>
<H2 class="top">Altro testo della pagina</H2>
...
<H2 class="bottom">Testo della pagina</H2>
<H2 class="bottom">Altro testo della pagina</H2>
```

# Pseudoclassi

- esempio: elementi anchor:
  - per default appartengono alla classe `link`, quelli visitati costituiscono la pseudoclasse `visited`, quelli attivi `active`, ...
- viene specificata all'interno dello stile seguita dai due punti
- esempio:

```
<STYLE>  
  a:link { text-decoration: none; }  
  a:visited { text-decoration: none; }  
</STYLE>
```

# Identificatori

- A tutti gli elementi è possibile assegnare l'attributo `ID`: il suo valore identifica univocamente quell'elemento all'interno del documento
- Questo attributo permette ai fogli di stile di trattare singole occorrenze di elementi



# Identificatori

Esempio:

...

```
<STYLE>
```

```
  #mioelem { font-weight:bold; }
```

```
</STYLE>
```

...

```
<p id="mioelem">testo 1</p>
```

```
<p>testo 2</p>
```

...

Il paragrafo "testo 1" verrà visualizzato in neretto

# Selettori CSS

- Selettore universale: \*
- Selettore di nome: *nome-elemento* (es. H1)
- Selettore di classe: *.nome-classe*
- Selettore di classi multiple:  
*.nome-classe1.nome-classe2....*
- Selettore di id: *#nome-id*
- Unione di selettori (,)
- Selettore di figli (>)
- Selettore di discendenti (sequenza con spazi)
- Selettori di fratelli (+), (~)

# Selettori CSS

- Selettori di pseudo-classi:
  - `:link`
  - `:visited`
  - `:hover`
  - `:focus`
  - `:first-child`
  - `:root`
  - `:nth-child()`

# Selettori CSS

- Selettori di pseudo-classi:
  - `:nth-last-child()`
  - `:last-child`
  - `:only-child`
  - `:nth-of-type()`
  - `:nth-last-of-type()`
  - `:first-of-type`
  - `:last-of-type`
  - `:only-of-type`

# Selettori CSS

- Selettori di pseudo-classi:
  - :not()
  - :is()
  - :has()
  - :empty
  - :target
  - :enabled
  - :disabled
  - :checked

# Selettori CSS

- Selettori di pseudo-classi:
  - :default
  - :valid
  - :invalid
  - :in-range
  - :out-of-range
  - :required
  - :optional
  - :read-only
  - :read-write

# Selettori CSS

- Selettori di pseudo-elementi:
  - `::first-letter`
  - `::first-line`
  - `::before` e `::after` (solitamente in combinazione con la proprietà `content`)
  - `::selection`
  - `::marker`
  - `::placeholder`

# Selettori CSS

- Selettori di attributi:
  - E[attribute]
  - E[attribute=value]
  - E[attribute~=value]
  - E[attribute|=value]
  - E[attribute^=value]
  - E[attribute\$=value]
  - E[attribute\*=value]



# Cascata

- Una pagina può importare più fogli di stile (e/o avere stili incorporati e/o avere stili inline)
- Tutte queste dichiarazioni possono entrare in conflitto
- Semplice esempio:

```
<style>
  h1 { color:green; }
  h1 { color:red; }
</style>
```

Di che colore sono gli elementi <h1> della pagina?

- Nei casi come questo, in cui le dichiarazioni usano lo stesso selettore, vince l'ultima dichiarazione
- Pertanto è rilevante l'ordine in cui le dichiarazioni sono scritte (o l'ordine in cui i fogli di stile esterni sono richiamati nella pagina)
- In generale, viene definito un **ordine di precedenza** tra le dichiarazioni CSS

# Specificità

- Altro esempio di conflitto:

```
<style>
  .c1 { color:red; }
  h1 { color:green; }
</style>
```

Se ho un elemento `<h1 class="c1">Ciao</h1>`, in che colore viene visualizzato?

- A questo elemento sono applicabili entrambe le dichiarazioni
- Vince la prima dichiarazione, perché è più specifica
- Ordine di specificità (decrescente):
  1. Dichiarazioni di id
  2. Dichiarazioni di classe
  3. Dichiarazioni di elemento
- Attenzione: la specificità vince sull'ordine in cui le dichiarazioni sono scritte (o l'ordine in cui i fogli di stile sono importati)

# !important

- L'ordine di precedenza può essere modificato dal flag **!important** che può essere assegnato ad una singola assegnazione di proprietà in una dichiarazione CSS
- Esempio:  

```
h1 { color: green;  
      font-style: italic !important; }
```
- Il flag **!important** rende l'assegnazione la più specifica di tutte...
  - ... a meno che non ci sia un'altra assegnazione **!important** in conflitto con questa!

# Ereditarietà

- Alcune proprietà CSS vengono ereditate (ricorsivamente) da elemento a sottoelemento nella pagina (ad es. font-style, color, ...)
- Altre non vengono ereditate (ad es. border, width, padding)
- È possibile modificare le regole di ereditarietà delle proprietà CSS mediante l'uso di valori speciali (inherit, initial, revert, revert-layer, unset)

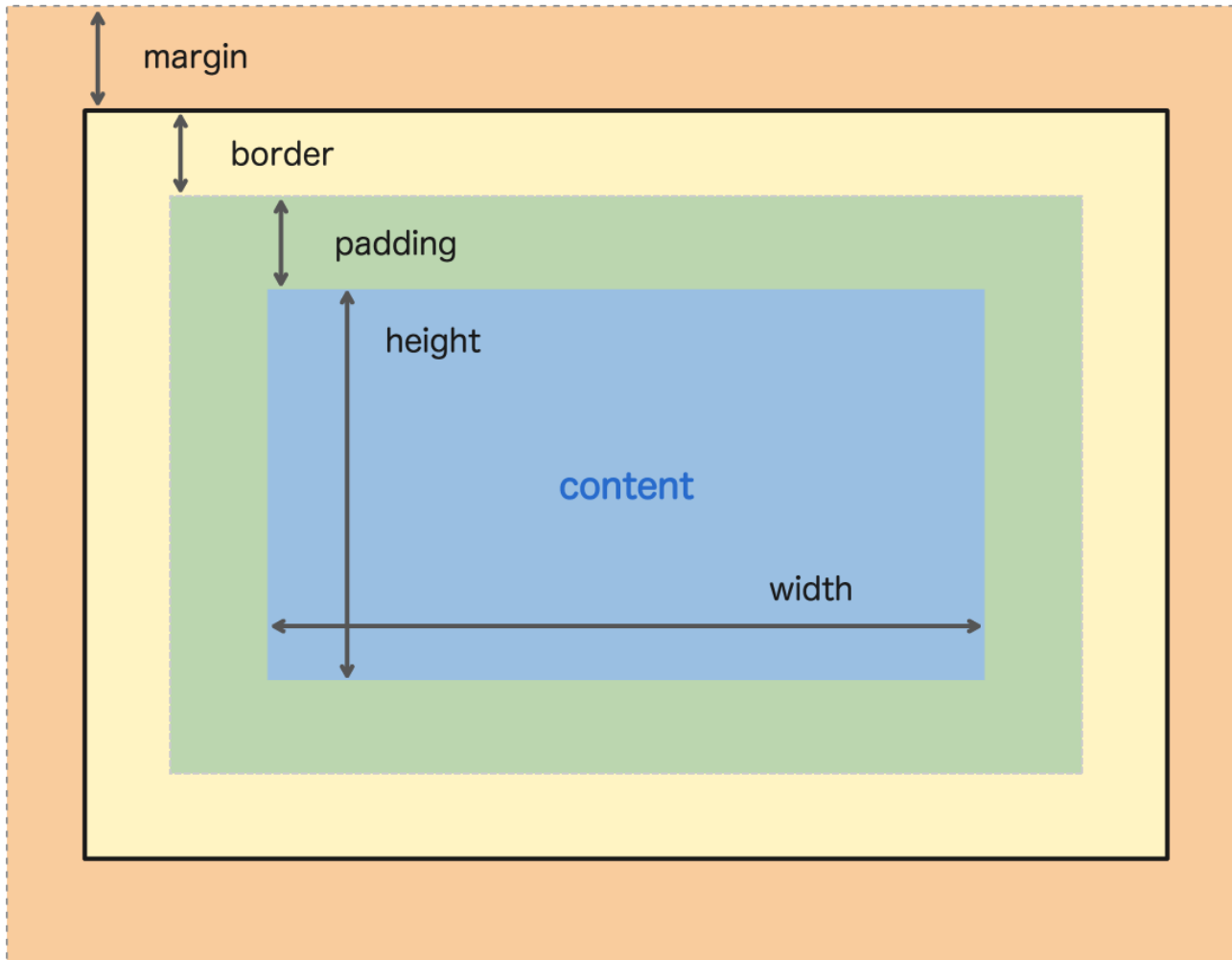
# Posizionamento

- CSS può essere usato per specificare il **posizionamento** di un elemento all'interno della pagina
- 5 valori per la proprietà **position**:
  - `static`
  - `relative`
  - `fixed`
  - `absolute`
  - `sticky`
- una volta specificato il tipo di posizionamento, è possibile usare altre proprietà, ad es. `left`, `right`, `top`, `bottom`, `z-index`, ecc.

# Altre proprietà utili

- `display` (`inline`, `block`, `inline-block`, ...)
- `border` / `margin` / `padding`
- `flex` / `grid` (le vedremo più avanti)
- `margin` / `padding`
- `overflow`
- `opacity`
- `scroll-behavior`
- `aspect-ratio`
- `accent-color`
- `filter`

# Bordi e margini



# Commenti

- Nei fogli di stile CSS è possibile inserire dei **commenti**
- I commenti sono multilinea e sono delimitati da `/*` e `*/`
- Esempio:

```
/* Questo è un commento */  
.myClass { color: red; }
```



# Funzioni

- funzioni relative ai colori ed effetti grafici in generale
  - es. `rgb()`, `translate()`, `blur()`, `rotate()`, `saturate()`, ecc.
- funzioni matematiche
  - es. `sin()`, `cos()`, `tan()`, `min()`, `max()`, `exp()`, `log()`, ecc.
- calcolo a partire da input CSS
  - es. `calc(100% - 100px)`
- ...e tante altre
  - `url()`, `var()`, `attr()`, ecc.

# Colori

- oltre al **codice esadecimale** (es. #FF0036), i colori possono essere indicati:
- tramite il proprio **nome** (vedi [elenco nomi riconosciuti](#))
- tramite le funzioni `RGB()` e `RGBA()`
  - canale rosso, giallo, blu e alfa (trasparenza)
- tramite le funzioni `HSL()` e `HSLA()`
  - *hue* (tonalità), *saturation* (saturazione), *lightness* (luminosità), *alpha*
- tante altre funzioni (es. `linear-gradient()`, `radial-gradient()`, `saturate()`)

# Variabili

- Come nei linguaggi di programmazione, possibile definire delle variabili da riutilizzare
- Il nome della variabile inizia per "--"
- La variabile viene richiamata tramite `var()`
- Esempio:

```
:root {  
  --my-color: #00df5e;  
}  
.my-class {  
  color: var(--my-color);  
}
```

- Attenzione allo "scope" in cui la variabile è stata definita!

# At-rules

- Le **at-rules** forniscono istruzioni al CSS su come comportarsi in determinate situazioni
- **@import** – importazione di altri file CSS
  - `@import url("stili.css");`
- **@media** – media queries per design responsivo
  - ```
@media (max-width: 600px) {  
    body {  
        background-color: lightblue;  
    }  
}
```
- **@font-face** – dichiarazione di font personalizzati
  - ```
@font-face {  
    font-family: "MioFont";  
    src: url("miofont.woff2") format("woff2");  
}
```

# At-rules

- **@keyframes** – animazioni CSS
  - @keyframes fadeIn {  
    from { opacity: 0; }  
    to { opacity: 1; }  
}
- **@layer** – organizzazione "a strati" delle regole CSS
- ...

# Layers

- Nei fogli di stile CSS è possibile creare degli strati (*layers*)
- L'ordine di precedenza dei vari layer è esplicitamente scritto nel foglio di stile stesso
- Questo è uno strumento potente ma è complesso da usare e complica ulteriormente la comprensione del funzionamento globale dei fogli di stile

# Proprietà CSS nel DOM

- Nel DOM ogni elemento HTML ha la proprietà `style`, che contiene tutte le proprietà CSS dichiarate per quell'elemento
- Il valore di tale proprietà `style` è un oggetto `CSSStyleDeclaration`

# Proprietà CSS nel DOM

- Proprietà dell'oggetto `CSSStyleDeclaration`:
  - `length`
  - `cssText`
  - `parentRule`
- Metodi dell'oggetto `CSSStyleDeclaration`:
  - `item(n)`
    - restituisce il nome della proprietà CSS di indice `n` dell'elemento
  - `getPropertyValue(nomeProprietàCSS)`
  - `setProperty(nomeProprietàCSS)`
  - `removeProperty(nomeProprietàCSS)`
  - `getPropertyPriority(nomeProprietàCSS)`



# Proprietà CSS nel DOM

- Nel DOM le proprietà CSS possono essere accedute come (sotto)proprietà della proprietà style dell'elemento:
- Esempio:  
`document.getElementById("abc").style.color="white";`  
assegna il colore bianco all'elemento con id uguale ad "abc"
- Altro esempio:  
`document.getElementsByTagName("p")[0].style.textAlign="right";`  
assegna allineamento destro al primo elemento paragraph (p) del documento

# Proprietà CSS nel DOM

- Se non si conoscono a priori le proprietà CSS definite per un elemento HTML, si possono estrarre a runtime
- Esempio:

```
var s = new Array();  
var x = document.getElementById("abc");  
for (var i = 0; i < x.style.length; i++)  
    s[i] = x.style.item(i);
```

memorizza nell'array s l'elenco delle proprietà CSS definite per l'elemento con id uguale ad "abc"

# Riferimenti

- Raccomandazioni ufficiali W3C:  
[w3.org/Style/CSS](http://w3.org/Style/CSS)
- Documentazione MDN Web Docs:  
[developer.mozilla.org/en-US/docs/Web/CSS](http://developer.mozilla.org/en-US/docs/Web/CSS)
- Tutorial CSS sul sito W3Schools:  
[w3schools.com/css](http://w3schools.com/css)